Model Regresi

Ir. Fandy Setyo Utomo, S.Kom., M.Cs., Ph.D

PSIFW074 – Pembelajaran Mesin

Program Studi Informatika - Universitas AMIKOM Purwokerto

Capaian Pembelajaran Praktikum:

Mahasiswa mampu membangun model Forecasting/Prediksi pada data numerik.

Software yang diperlukan dalam pembelajaran:

1. Python
2. Pandas (https://pandas.pydata.org/)
3. Seaborn (https://seaborn.pydata.org/)
4. Matplotlib (https://matplotlib.org/)
5. Scikit-learn/sklearn (https://scikit-learn.org/stable/)
6. Spyder (*Scientific Python Development Environment*)

Tahapan Praktikum:

1. Import seluruh pustaka yang dibutuhkan untuk praktikum pada IDE Spyder:

```
#import libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.linear_model import LinearRegression
```

2. Import dataset dengan menuliskan perintah berikut pada IDE Spyder:

```
#import dataset
dataset=pd.read_csv('california_housing_dataset.csv')
df=pd.DataFrame(dataset)
```

Catatan:

median_house_value menjadi target variabel dalam model prediksi harga rumah.

3. Penanganan *data pre-processing* dengan menuliskan perintah berikut pada IDE Spyder:

```
#identifikasi missing values
print(df.info())
mv=df.isna().sum()
print('\nJumlah missing value tiap kolom:\n', mv)
```

```
5.
6.  '''
7.  Solusi Categorical Data:
8.  Pengodean kolom ocean_proximity dengan LabelEncoder
9.  '''
10. label_encoder_x= LabelEncoder()
11. df['ocean_proximity']= label_encoder_x.fit_transform(df['ocean_proximity'])
12.
13. #identifikasi korelasi antar variabel
14. correlation = df.corr(method='pearson')
15.
16. '''
17. Solusi missing value:
18. Isi missing value dengan nilai median
19. '''
20. #membagi dataframe menjadi 20 blok berdasarkan 'households'
21. df['block'] = pd.cut(df['households'], bins=20, labels=False)
22.
23. #fungsi untuk mengisi NaN dengan median dari setiap blok
24. def fill_nan_with_block_median(group):
25.     return group.fillna(group.median())
26.
27. #mengisi NaN pada kolom 'total_bedrooms' dengan median dari
28. #blok 'total_bedrooms' dari blok record tersebut berasal
29. df['total_bedrooms'] =
      df.groupby('block')['total_bedrooms'].transform(fill_nan_with_block_median)
30.
31. #menghapus kolom 'block' setelah proses pengisian NaN selesai
32. df.drop(columns=['block'], inplace=True)
33.
34. #scatter plot:'median_income' vs 'median_house_value'
35. sns.scatterplot(x='median_income', y='median_house_value', data=df)
36. plt.xlabel('Median Income')
37. plt.ylabel('Median House Value')
38. plt.title('Income vs House Value')
39. plt.show()
40.
41. '''
42. Solusi feature scaling dengan MinMaxScaler
43. '''
44. scaler = MinMaxScaler()
45. df[['latitude', 'longitude']] = scaler.fit_transform(df[['latitude', 'longitude']])
```

4. Ekstraksi variabel independen dan dependen dari *DataFrame* dengan menuliskan perintah berikut pada IDE Spyder:

```
1.  #ekstraksi variabel independen
2.  X = df.drop(columns=['median_house_value'])
3.
4.  #ekstraksi variabel dependen
```

```
y = df['median_house_value']
```

5. Seleksi fitur dapat dituliskan sebagai berikut pada IDE Spyder:

```
'''
seleksi fitur menggunakan chi-square
'''
# Memilih 7 fitur terbaik
selector = SelectKBest(score_func=chi2, k=7)
X_selected = selector.fit_transform(X, y)
```

6. Data spliting dapat dituliskan sebagai berikut pada IDE Spyder:

```
'''
Data spliting
'''
#pembagian dataset menjadi training dan test set
X_train, X_test, y_train, y_test = train_test_split(X_selected, y, test_size = 0.2,
  random_state = 0)
```

7. Implementasi Simple Linear Regression dapat dituliskan sebagai berikut pada IDE Spyder:

```
'''
Evaluasi model Regresi
'''
def evaluate_model(model, X_test, y_test):
    y_pred = model.predict(X_test)
    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    print("Mean Squared Error:", mse)
    print("R-squared:", r2)

  return y_pred

'''
Model Simple Linear Regression
'''
#membuat model
simple_lr = LinearRegression()

#melatih model dengan mengambil value dari kolom median_income (X_train)
simple_lr.fit(X_train[:, 5].reshape(-1, 1), y_train)

#evaluasi model dengan mengambil value dari kolom median_income (X_test)
print("Simple Linear Regression:")
y_pred = evaluate_model(simple_lr, X_test[:, 5].reshape(-1, 1), y_test)
```

```
#plot hasil
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred, alpha=0.5)
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.title('Simple Linear Regression: Actual vs. Predicted Values for Median House Value')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red')
plt.show()
```

8. Implementasi Multiple Linear Regression dapat dituliskan sebagai berikut pada IDE Spyder:

```
'''
Multiple Linear Regression
'''
#membuat model
multiple_lr = LinearRegression()

#melatih model
multiple_lr.fit(X_train, y_train)

#evaluasi model
print("Multiple Linear Regression:")
y_pred = evaluate_model(multiple_lr, X_test, y_test)

#plot hasil
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred, alpha=0.5)
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.title('Multiple Linear Regression: Actual vs. Predicted Values for Median House Value')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red')
plt.show()
```

KODE PROGRAM GABUNGAN

Berikut ini merupakan kode program gabungan, mulai dari import pustaka yang dibutuhkan untuk menyelesaikan studi kasus hingga evaluasi model regresi:

```
#import libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.linear_model import LinearRegression

#import dataset
dataset=pd.read_csv('california_housing_dataset.csv')
df=pd.DataFrame(dataset)

#identifikasi missing values
print(df.info())
mv=df.isna().sum()
print('\nJumlah missing value tiap kolom:\n', mv)

'''
Solusi Categorical Data:
Pengodean kolom ocean_proximity dengan LabelEncoder
'''
label_encoder_x= LabelEncoder()
df['ocean_proximity']= label_encoder_x.fit_transform(df['ocean_proximity'])

#identifikasi korelasi antar variabel
correlation = df.corr(method='pearson')

'''
Solusi missing value:
Isi missing value dengan nilai median
'''
#membagi dataframe menjadi 20 blok berdasarkan 'households'
df['block'] = pd.cut(df['households'], bins=20, labels=False)

#fungsi untuk mengisi NaN dengan median dari setiap blok
def fill_nan_with_block_median(group):
    return group.fillna(group.median())

#mengisi NaN pada kolom 'total_bedrooms' dengan median dari
#blok 'total_bedrooms' dari blok record tersebut berasal
df['total_bedrooms'] =
df.groupby('block')['total_bedrooms'].transform(fill_nan_with_block_median)

#menghapus kolom 'block' setelah proses pengisian NaN selesai
```

```
df.drop(columns=['block'], inplace=True)

#scatter plot:'median_income' vs 'median_house_value'
sns.scatterplot(x='median_income', y='median_house_value', data=df)
plt.xlabel('Median Income')
plt.ylabel('Median House Value')
plt.title('Income vs House Value')
plt.show()

'''
Solusi feature scaling dengan MinMaxScaler
'''
scaler = MinMaxScaler()
df[['latitude', 'longitude']] = scaler.fit_transform(df[['latitude', 'longitude']])

#ekstraksi variabel independen
X = df.drop(columns=['median_house_value'])

#ekstraksi variabel dependen
y = df['median_house_value']

'''
seleksi fitur menggunakan chi-square
'''
# Memilih 7 fitur terbaik
selector = SelectKBest(score_func=chi2, k=7)
X_selected = selector.fit_transform(X, y)

'''
Data spliting
'''
#pembagian dataset menjadi training dan test set
X_train, X_test, y_train, y_test = train_test_split(X_selected, y, test_size = 0.2,
random_state = 0)

'''
Evaluasi model Regresi
'''
def evaluate_model(model, X_test, y_test):
    y_pred = model.predict(X_test)
    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    print("Mean Squared Error:", mse)
    print("R-squared:", r2)

    return y_pred

'''
Model Simple Linear Regression
'''
#membuat model
simple_lr = LinearRegression()
```

```python

#melatih model dengan mengambil value dari kolom median_income (X_train)
simple_lr.fit(X_train[:, 5].reshape(-1, 1), y_train)

#evaluasi model dengan mengambil value dari kolom median_income (X_test)
print("Simple Linear Regression:")
y_pred = evaluate_model(simple_lr, X_test[:, 5].reshape(-1, 1), y_test)

#plot hasil
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred, alpha=0.5)
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.title('Simple Linear Regression: Actual vs. Predicted Values for Median House Value')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red')
plt.show()

'''
Multiple Linear Regression
'''
#membuat model
multiple_lr = LinearRegression()

#melatih model
multiple_lr.fit(X_train, y_train)

#evaluasi model
print("Multiple Linear Regression:")
y_pred = evaluate_model(multiple_lr, X_test, y_test)

#plot hasil
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred, alpha=0.5)
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.title('Multiple Linear Regression: Actual vs. Predicted Values for Median House Value')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red')
plt.show()
```

Tugas Praktikum:

1. Berdasarkan hasil evaluasi *Mean Squared Error* dan *R-squared* terhadap kedua model regresi. Jelaskan mana model regresi yang terbaik dari eksperimen yang sudah dilakukan?
2. Pada model *Multiple Linear Regression* gunakan 2 variabel bebas/independen untuk memprediksi '*households*' (*Number of households in the area*). **Petunjuk**: Untuk

menentukan 2 variabel bebas yang akan digunakan, lihat hasil analisis pearson correlation. Variabel mana saja yang memiliki nilai korelasi tertinggi dengan '*households*'.

3. Gunakan model *Support Vector Regression* (SVR) untuk memprediksi '*median_house_value*' menggunakan seluruh variabel bebas/independen. Gunakan kernel: 'linear', 'poly', 'rbf' dalam eksperimen. Berdasarkan metrik evaluasi *Mean Squared Error* dan *R-squared*, jelaskan kernel mana yang memiliki hasil terbaik pada model SVR?

   Petunjuk: https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVR.html
4. Gunakan model *Polynomial Regression* untuk memprediksi '*median_house_value*' menggunakan seluruh variabel bebas/independen. Gunakan derajat/orde polinomial (*degree*) = 2 dalam eksperimen. Berdasarkan metrik evaluasi *Mean Squared Error* dan *R-squared*, jelaskan model regresi mana (*Polynomial Regression*, SVR, atau *Multiple Linear Regression*) yang memiliki hasil terbaik? Petunjuk: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.PolynomialFeatures.html

Laporan Praktikum:

Laporan praktikum yang berisi modifikasi kode program yang telah diubah berdasarkan tugas praktikum diunggah melalui halaman: https://forms.gle/znCaFz4R71FDKVfX7 paling lambat H-1 sebelum perkuliahan minggu depan. Kolom "Tugas Pertemuan" pilih pertemuan "4."